

Blockchain-Based Library Lending Tracker

Technical Report · Individual Formative Assignment 1 · Module: Introduction to Blockchain Development

Author: Joseph Nishimwe · **Date:** 21 September 2026

Repository: https://github.com/josep-prog/formative_introduction_blockchain

1. Introduction

A paper or spreadsheet lending log can be quietly edited. A librarian can mark a lost book as "returned", or a borrower can dispute what the log says, and nothing in the record itself shows that it was changed. This project builds a small library lending tracker in C where every borrow and return is stored as a block in a blockchain. Each block carries the SHA-256 hash of the block before it and an ECDSA digital signature, so changing any past record can be detected.

The program loads a book registry and a member registry from text files, refuses any lending action that names an unknown book or member, and offers a command-line menu to borrow, return, view records, validate the chain and demonstrate tamper detection. The chain and the signing key are saved to disk, so records survive a restart.

2. Project Files

File	Purpose
src/main.c	Menu and program flow. Loads the registries, key and chain at start-up, checks IDs, calls the blockchain functions, and saves the chain after each successful borrow or return.
src/blockchain.c / .h	The <code>Block</code> structure; creating the genesis, borrow and return blocks; hashing; finding a book's active loan; validating the chain; saving and loading the chain file.
src/crypto.c / .h	OpenSSL wrappers: ECDSA key generation, key save/load (PEM), signing and signature verification.
src/registry.c / .h	The <code>Book</code> and <code>Member</code> structures; loading <code>books.txt</code> and <code>members.txt</code> ; exact-match lookup by ID.
data/books.txt , data/members.txt	The registries (5 books, 5 members).
Makefile	Builds the <code>library</code> binary with <code>-Wall -Wextra -std=c11</code> and links OpenSSL.
data/chain.txt , data/key.pem	Created at run time: the saved blockchain and the signing key. Both are listed in <code>.gitignore</code> .

3. System Design Diagram

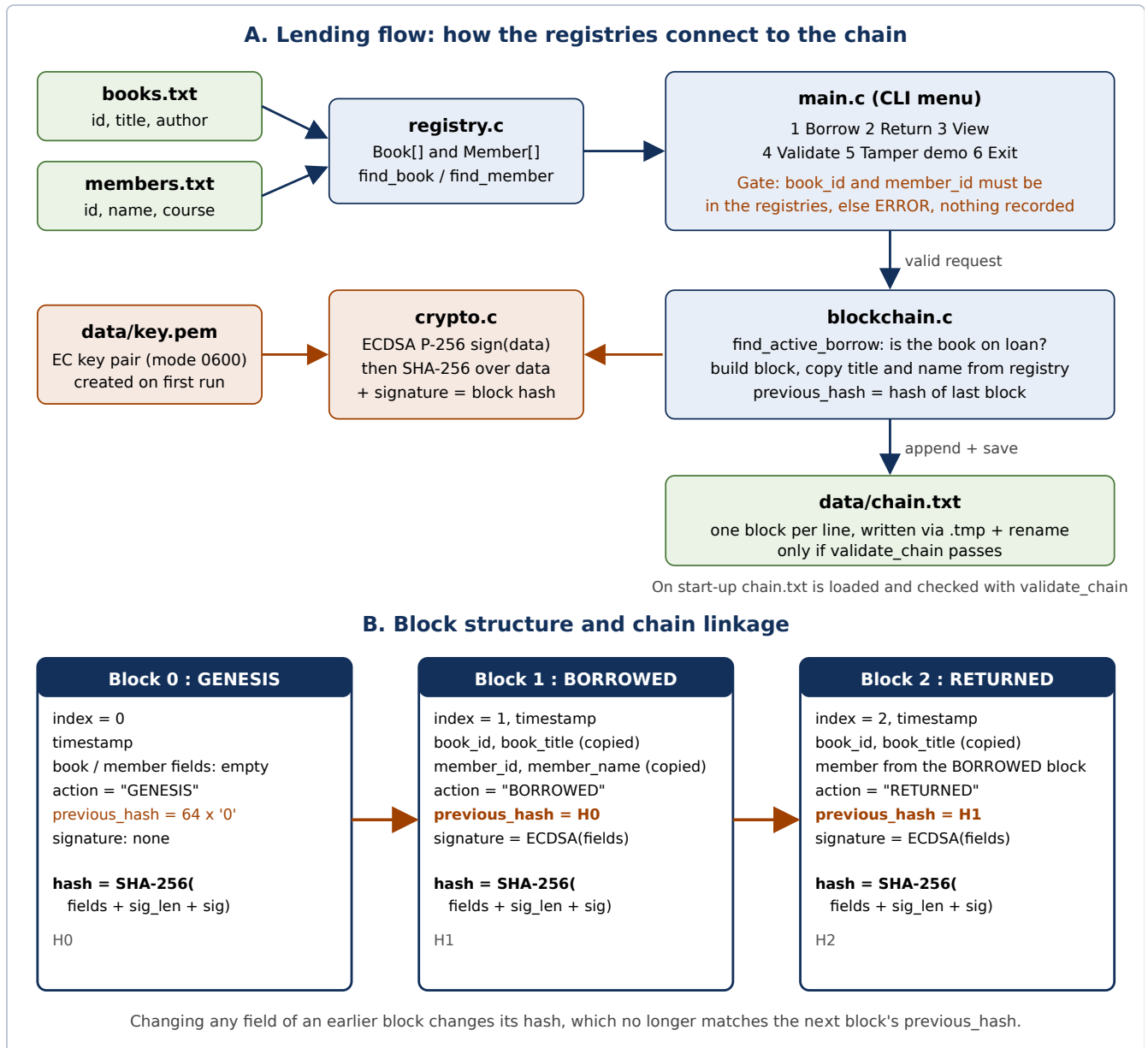


Figure 1. (A) Registries gate every lending action before a block is built, signed, hashed and saved. (B) Each block stores the hash of the block before it.

4. Description of the Blockchain Implementation

4.1 Block structure

Each block is the `Block` struct in `blockchain.h`. It has every field the assignment asks for. One extra field, `signature_length`, is needed because an ECDSA signature does not always have the same length (see Challenges).

Field	Type	Description
index	int	Position in the chain (0 = genesis).
timestamp	time_t	Unix time when the block was created.
book_id, book_title	char[20], char[80]	ID checked against <code>books.txt</code> ; the title is copied from the registry at that moment.
member_id, member_name	char[20], char[50]	ID checked against <code>members.txt</code> ; the name is copied from the registry.
action	char[10]	<code>BORROWED</code> or <code>RETURNED</code> (plus <code>GENESIS</code> for block 0).
previous_hash	char[65]	Hash of the previous block (64 zeros for the genesis block).
signature, signature_length	unsigned char[72], unsigned int	ECDSA signature of the block data, and its real length in bytes.
hash	char[65]	SHA-256 of all the fields above combined, as 64 hex characters.

4.2 Operations

- **Load registries.** `load_books()` and `load_members()` read the two files into arrays of `Book` and `Member`. If either file is missing or yields no records, an error is printed and the program exits.
- **Genesis block.** If no saved chain exists, block 0 is created with `previous_hash` set to 64 zeros.
- **Borrow.** Both IDs are looked up with an exact `strcmp` match. If either is missing the program prints `ERROR: Book or Member not found` and records nothing. Otherwise `find_active_borrow()` scans the chain from newest to oldest; if the latest event for that book is `BORROWED`, the borrow is refused. If the book is free, the block is built, signed, hashed and appended.
- **Return.** The user gives only the book ID. If the latest event for the book is not `BORROWED`, an error is printed. Otherwise a `RETURNED` block is created, using the member from the original `BORROWED` block.
- **Validate chain.** For every block, `validate_chain()` recalculates the hash and compares it with the stored one, checks that `previous_hash` equals the previous block's hash, and verifies the ECDSA signature of every block after the genesis block.
- **View records.** Prints each block with book title, member name, action, timestamp, hash, previous hash, and whether the signature is `VALID` or `INVALID`.
- **Tamper detection.** Menu option 5 changes the title stored in block 1 in memory, without re-signing or re-hashing, then runs the validation.

5. Security Mechanisms

5.1 SHA-256 hashing

The block hash is SHA-256 (OpenSSL) over the text

`index|timestamp|book_id|book_title|member_id|member_name|action|previous_hash`, followed by the signature length and the signature bytes. Because the hash also covers `previous_hash`, each block depends on every block before it. Editing an old block changes its hash, breaks the link stored in the next block, and so invalidates the rest of the chain.

5.2 ECDSA digital signatures (authentication)

Every borrow and return block is signed with ECDSA on the P-256 curve (SHA-256 digest) using OpenSSL's EVP interface. The signature covers the same eight data fields that go into the hash. Signatures are checked in `validate_chain()` and again for each block when records are viewed. A person

who edits a block and recalculates its hash still cannot produce a valid signature without the private key, so a forged block is still caught.

5.3 Key management

On the first run a key pair is generated and saved to `data/key.pem` in PEM format with file mode `0600`. Later runs load the same key, so old signatures stay verifiable after a restart. The key file is in `.gitignore` and is never committed.

5.4 Input validation

IDs are read with a width-limited `%19s` so they cannot overflow the 20-byte fields, and are matched exactly, so `BK001EXTRA` is not accepted as `BK001`. The chain-file parser rejects lines with the wrong number of fields, over-long fields, hashes that are not 64 characters, or non-hex signatures.

6. Data Persistence Approach

- **Registries** are plain text (`books.txt`, `members.txt`) read at every start-up.
- **The chain** is saved to `data/chain.txt` after every successful borrow or return, one block per line:
`index|timestamp|book_id|book_title|member_id|member_name|action|previous_hash|signature_hex|hash`. It is plain text on purpose so it can be opened and edited to test tamper detection.
- **Safe writes.** The file is written to `chain.txt.tmp` and then renamed over `chain.txt`, so a crash cannot leave a half-written chain.
- **Only valid chains are saved.** `save_or_warn()` validates the chain first. The tamper demo therefore cannot end up on disk.
- **On start-up** the saved chain is loaded and validated. If it is invalid, a warning is printed. If there is no file, a new chain with a genesis block is started. If the file exists but cannot be parsed, the program stops instead of overwriting it.
- **The signing key** is stored in `data/key.pem` (section 5.3).

7. Error Handling Strategy

Situation	Behaviour
<code>books.txt</code> or <code>members.txt</code> missing	Error naming the file; program exits with status 1.
Registry file empty or badly formatted	Error message; program exits.
Unknown book or member ID on borrow	ERROR: Book or Member not found; no block created.
Unknown book ID on return	Same error; no block created.
Borrowing a book already on loan	ERROR: This book is already on loan.
Returning a book that is not on loan	ERROR: This book is not currently on loan.
Chain full (1000 blocks)	ERROR: Blockchain is full.
Saved chain fails validation at start-up	Warning that it may have been tampered with.
Chain file exists but is unreadable	Error; program stops rather than overwrite it.
Chain fails validation when saving	Warning; file is not written.
Key file missing or invalid	A new key pair is generated and saved.
Non-numeric menu input	Input is cleared and the menu is shown again.
Input stream closed (EOF)	Program exits cleanly instead of looping.

8. Build and Run

Requirements: a C compiler (`gcc`), `make` , and the OpenSSL development library.

```
sudo apt install gcc make libssl-dev
git clone https://github.com/josep-prog/formative_introduction_blockchain.git
cd formative_introduction_blockchain
make clean && make
./library
```

The program must be run from the project folder because it reads `data/books.txt` and `data/members.txt` by relative path. To start with a fresh chain, delete `data/chain.txt` .

Books (<code>books.txt</code>)	Members (<code>members.txt</code>)
BK001 The Money Trap: Lost Illusions Inside the Tech Bubble (Alok Sama)	ALU001 Irakoze Jean (BSE)
BK002 Wars Guns & Votes: Democracy in Dangerous Places (Paul Collier)	ALU002 Joseph Habimana (BSE)
BK003 Sustainable Leadership (Clarke Murphy)	ALU003 Uwase Diane (BEL)
BK004 Ancient Philosophy: The Fundamentals (Daniel W. Graham)	ALU004 Manzi Eric (BSE)
BK005 Gulliver's Travels and Other Writings (Jonathan Swift)	ALU005 Mukamana Alice (BEL)

9. Screenshots of Application Execution

The figures below are the program's real output, captured from the current build with a fresh `data/` folder. The repeated menu is shown as `[menu]` to save space.

9.1 First run: registries loaded, key generated, genesis block, first borrow

```
Loaded 5 books and 5 members.
Digital signing key generated and saved to data/key.pem.
Started a new blockchain.

[menu]
Book ID   : Member ID: Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean.

[menu]
Goodbye.
```

Figure 2. On the first run the key is generated and saved and a new blockchain is started.

9.2 Persistence, invalid IDs and lending rules (second run)

```
Loaded 5 books and 5 members.
Digital signing key loaded from data/key.pem.
Loaded 2 blocks from data/chain.txt.

[menu]
Book ID   : Member ID: ERROR: This book is already on loan.

[menu]
Book ID   : Member ID: ERROR: Book or Member not found

[menu]
Book ID   : Member ID: ERROR: Book or Member not found

[menu]
Book ID: ERROR: This book is not currently on loan.

[menu]
Book ID: ERROR: Book or Member not found

[menu]
Goodbye.
```

Figure 3. The key and the 2-block chain are loaded from disk. Borrowing an on-loan book, unknown book, unknown member, returning a book that is not on loan and returning an unknown book are all rejected.

9.3 Return, view records and validate

```
Loaded 5 books and 5 members.
Digital signing key loaded from data/key.pem.
Loaded 2 blocks from data/chain.txt.

[menu]
Book ID: Returned 'The Money Trap: Lost Illusions Inside the Tech Bubble'.

[menu]

-----
Block #0 [GENESIS]
  Time   : Mon Sep 21 13:10:39 2026
  Hash   : eea076c1a854a2480a2c9ec73a6c63f4d9b5bb22cc72924446c2660cf6104a7d
  Prev   : 0000000000000000000000000000000000000000000000000000000000000000
  Signature: n/a (genesis block)

-----
Block #1 [BORROWED]
  Book    : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
  Member  : Irakoze Jean (ALU001)
  Time    : Mon Sep 21 13:10:39 2026
  Hash    : d8ee5198b1d2212d22719058a4cd971dabe2ba1144b8b5f1687363763997d655
  Prev    : eea076c1a854a2480a2c9ec73a6c63f4d9b5bb22cc72924446c2660cf6104a7d
  Signature: VALID (70 bytes)

-----
Block #2 [RETURNED]
  Book    : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
  Member  : Irakoze Jean (ALU001)
  Time    : Mon Sep 21 13:10:39 2026
  Hash    : 85ade0722c50e5a98ef51051e4d69d72892cb9fa514e2dbd482e5b2007be2d69
  Prev    : d8ee5198b1d2212d22719058a4cd971dabe2ba1144b8b5f1687363763997d655
  Signature: VALID (70 bytes)

[menu]
Blockchain is VALID - no tampering detected.

[menu]
Goodbye.
```

Figure 4. The return block keeps the original borrower. All signatures are VALID and the chain validates.

9.4 Tamper-detection demo

```
Loaded 5 books and 5 members.
Digital signing key loaded from data/key.pem.
Loaded 3 blocks from data/chain.txt.

[menu]
Book ID : Member ID: Borrowed 'Sustainable Leadership' for Uwase Diane.

[menu]
This will edit Block #1's stored book title in memory
(the saved file is left untouched), WITHOUT re-signing or
re-hashing it - exactly what an attacker trying to rewrite
history would do.
Block #1's book_title has been altered.
Running chain validation...
Blockchain is INVALID - tampering detected, as expected.
(Restart the program to reload the clean chain from disk.)

[menu]
Blockchain is INVALID - tampering detected!

[menu]
Goodbye.
```

Figure 5. Block 1's title is changed in memory without re-hashing or re-signing; validation reports *INVALID* and keeps doing so.

9.5 Restart after the demo, then editing the saved file by hand

```
Loaded 5 books and 5 members.
Digital signing key loaded from data/key.pem.
Loaded 4 blocks from data/chain.txt.

[menu]
Blockchain is VALID - no tampering detected.

[menu]
Goodbye.
```

Figure 6. The tamper demo never reached the disk, so the reloaded chain is *VALID*.

```
Loaded 5 books and 5 members.
Digital signing key loaded from data/key.pem.
Loaded 4 blocks from data/chain.txt.
WARNING: the saved blockchain is INVALID - it may have been tampered with!

[menu]
Blockchain is INVALID - tampering detected!

[menu]
Goodbye.
```

Figure 7. After changing "Irakoze Jean" to "Someone Else" in `data/chain.txt` with a text editor, the program warns at start-up and validation fails.

10. Challenges Encountered and Solutions

Challenge	Solution
Validation altered the chain it was checking. Recalculating a hash in place overwrote the stored hash, so a tampered block could appear fixed.	<code>validate_chain()</code> recalculates the hash on a copy of each block and compares it with the stored value, so validation never modifies the chain.
ECDSA signatures are not a fixed size. In my runs the same code produced signatures of 69, 70, 71 and 72 bytes.	The signature buffer is 72 bytes (the maximum) and the real size is stored in <code>signature_length</code> . That length is included in the hash and saved in the chain file.
Signatures could not be checked after a restart because a new key was generated on every run.	The key pair is saved to <code>data/key.pem</code> (mode 0600) and loaded on later runs.
Adding a saved chain risked the tamper demo, or a crash, corrupting the file.	The demo edits memory only. <code>save_or_warn()</code> refuses to save an invalid chain, and writing goes through a temporary file and <code>rename()</code> . An unreadable chain file stops the program instead of being replaced.
Reading simple comma-separated registry lines without <code>strtok</code> and without overflowing fixed-size fields.	<code>fscanf</code> scan sets with width limits (for example <code>%19[^,]</code>) leave room for the terminating null byte.
Knowing whether a book is on loan without a second data structure, and who to name on a return.	<code>find_active_borrow()</code> scans the chain backwards to the book's latest event. The blockchain is the single source of truth, and the return block copies the member from that borrow block.
Loops or crashes on bad terminal input.	The menu detects EOF and exits, and clears non-numeric input before showing the menu again.

11. Current Limitations

- There is no login or role system, so anyone who can run the program can borrow and return. The signing key identifies the system, not an individual librarian.
- `data/key.pem` is not encrypted. Anyone who can read it can sign new blocks, and one key signs everything.
- `OVERDUE` is allowed by the block structure but the program only creates `BORROWED` and `RETURNED` blocks.
- The whole chain file is rewritten after every transaction, which is fine for a small library but not a large one. The chain is capped at 1000 blocks.
- Registry loading stops at the first malformed line, so records after it are ignored.
- `validate_chain()` does not check that block 0 is a genuine genesis block or that indexes are consecutive.
- Deleting the last blocks from `chain.txt` leaves a valid shorter chain, because a local chain has no other copies to compare with.

12. Conclusion

The project applies the core blockchain idea to a library lending problem. The registries decide what is valid, the blockchain records the history, SHA-256 links the blocks, ECDSA signatures authenticate each action, and validation detects any change to past records. Saving the chain and key to disk makes the history survive restarts. Adding a login and an `OVERDUE` workflow would be the next steps.